Testes de Integridade — Petly Club

ERP Petly Club · Grupo 7 · Modelagem de Dados · PostgreSQL 16 · executados em 27/10/2026

Objetivo e ambiente

Este documento verifica se o banco protege as regras de negócio por conta própria, mesmo que a aplicação tente gravar algo errado. Cada teste segue o formato **regra de negócio → restrição → comando → resultado**. Os comandos da parte 1 foram feitos para violar uma regra e precisam ser **recusados**; os da parte 2 são UPDATE e DELETE válidos, que precisam ser **aceitos** sem quebrar nenhuma regra.

> **Ambiente:** PostgreSQL 16. A base foi criada com `petly_club_ddl_limpo.sql` e carregada com `Massa_de_Dados_Completa_-_ERP_Petly_Club.sql`. Os comandos estão em `testes_integridade.sql` e podem ser repetidos com `psql -d petly_club -a -f testes_integridade.sql`. As imagens mostram a saída real do psql de cada comando. Os testes da parte 2 rodam dentro de BEGIN ... ROLLBACK, então a base continua igual depois da execução (conferido: 28 movimentações e 6 pedidos antes e depois).

Resumo

Teste	O que tenta fazer	Restrição	Esperado	Resultado
T01	PK duplicada	`pk_categoria (PRIMARY KEY)`	Recusar	**Recusado ✔**
T02	ID digitado manualmente	`id_cliente GENERATED ALWAYS AS IDENTITY`	Recusar	**Recusado ✔**
T03	CPF repetido	`uk_cliente_cpf (UNIQUE)`	Recusar	**Recusado ✔**
T04	Código de barras repetido	`uk_produto_barras (UNIQUE)`	Recusar	**Recusado ✔**
T05	Pedido de cliente inexistente	`fk_pedido_cliente (FOREIGN KEY)`	Recusar	**Recusado ✔**
T06	Venda saindo do lote de outro produto	`fk_mov_lote: FK composta (id_lote, id_produto) → lote`	Recusar	**Recusado ✔**
T07	Excluir cliente que tem pedidos	`fk_pedido_cliente ON DELETE RESTRICT`	Recusar	**Recusado ✔**
T08	Produto sem categoria	`produto.id_categoria NOT NULL`	Recusar	**Recusado ✔**
T09	Preço zero	`ck_produto_preco CHECK (preco > 0)`	Recusar	**Recusado ✔**
T10	Loja física com entrega	`ck_pedido_canal_recebimento (CHECK)`	Recusar	**Recusado ✔**
T11	Venda no caixa sem funcionário	`ck_pedido_func_fisica (CHECK)`	Recusar	**Recusado ✔**
T12	Cancelamento no caixa sem autorização	`ck_pedido_cancel_fisica (CHECK)`	Recusar	**Recusado ✔**
T13	Perda sem autorizador	`ck_mov_autorizacao (CHECK)`	Recusar	**Recusado ✔**
T14	Perda sem motivo	`ck_mov_motivo (CHECK)`	Recusar	**Recusado ✔**
T15	Crédito em 4 vezes	`ck_pag_parcelas CHECK (numero_parcelas BETWEEN 1 AND 3)`	Recusar	**Recusado ✔**

T16	Débito parcelado	`ck_pag_parcelas_credito (CHECK)`	Recusar	**Recusado ✔**
T17	Conta recebida sem data	`ck_cr_recebimento (CHECK)`	Recusar	**Recusado ✔**
T18	Mesmo produto duas vezes no pedido	`pk_item_pedido PRIMARY KEY (id_pedido, id_produto)`	Recusar	**Recusado ✔**
T19	Mesmo produto duas vezes na compra	`pk_item_compra PRIMARY KEY (id_compra, id_produto)`	Recusar	**Recusado ✔**
T20	Alterar uma movimentação de estoque	`trg01_mov_imutavel (gatilho BEFORE UPDATE OR DELETE)`	Recusar	**Recusado ✔**
T21	Vender sem saldo	`trg02_mov_saldo (gatilho BEFORE INSERT)`	Recusar	**Recusado ✔**
T22	Entrada em lote vencido	`trg03_mov_validade (gatilho BEFORE INSERT)`	Recusar	**Recusado ✔**
T23	Pagar mais que o valor da parcela	`trg08_pagconta_valida (gatilho BEFORE INSERT)`	Recusar	**Recusado ✔**
T24	Duas vendas simultâneas da última unidade	`trg02_mov_saldo (FOR UPDATE)`	Aceitar 1, recusar 1	**Correto ✔**
U01	UPDATE: reajuste de preço não altera vendas antigas	`item_pedido.preco_unitario guarda o preço do momento da venda`	Aceitar	**Aceito ✔**
U02	UPDATE: pedido entregue	`ck_pedido_status_fluxo e ck_entrega_data`	Aceitar	**Aceito ✔**
U03	INSERT + gatilho: pagamento que quita a parcela	`trg08_pagconta_status (gatilho AFTER INSERT)`	Aceitar	**Aceito ✔**
D01	DELETE: categoria sem produtos	`fk_produto_categoria ON DELETE RESTRICT (só bloqueia se houver produto)`	Aceitar	**Aceito ✔**

Resultado geral: os 23 comandos inválidos foram recusados pelo banco, o teste de concorrência terminou com saldo 0 (e não −1), e os 4 comandos válidos foram aceitos com o efeito esperado.

Parte 1 — Comandos que o banco deve recusar

1. Chave primária e unicidade

[T01] PK duplicada

Regra de negócio: Cada categoria é identificada por um único id (identificação única).

Restrição: `pk_categoria (PRIMARY KEY)`

Comando e resultado:

psql — petly_club (PostgreSQL 16) — executado em 27/10/2026

```
petly_club=# INSERT INTO categoria (id_categoria, nome)
petly_club-# OVERRIDING SYSTEM VALUE
petly_club-# VALUES (1, 'Categoria Repetida');
linha 3: ERROR:  duplicate key value violates unique constraint "pk_categoria"
DETAIL:  Key (id_categoria)=(1) already exists.
```

Por que foi recusado: O id 1 já pertence à categoria 'Racoes e Alimentos'. **O banco recusou corretamente.**

[T02] ID digitado manualmente

Regra de negócio: O identificador é gerado pelo sistema e não muda (GENERATED ALWAYS AS IDENTITY).

Restrição: `id_cliente GENERATED ALWAYS AS IDENTITY`

Comando e resultado:

psql — petly_club (PostgreSQL 16) — executado em 27/10/2026

```
petly_club=# INSERT INTO cliente (id_cliente, nome, cpf)
petly_club-# VALUES (50, 'Cliente Teste', '11111111111');
linha 2: ERROR:  cannot insert a non-DEFAULT value into column "id_cliente"
DETAIL:  Column "id_cliente" is an identity column defined as GENERATED ALWAYS.
HINT:  Use OVERRIDING SYSTEM VALUE to override.
```

Por que foi recusado: O banco não aceita que a aplicação escolha o id; ele é sempre gerado. **O banco recusou corretamente.**

[T03] CPF repetido

Regra de negócio: RN18: o mesmo CPF não pode ser cadastrado duas vezes.

Restrição: `uk_cliente_cpf (UNIQUE)`

Comando e resultado:

psql — petly_club (PostgreSQL 16) — executado em 27/10/2026

```
petly_club=# INSERT INTO cliente (nome, cpf, email)
petly_club-# VALUES ('Maria S. Copia', '12345678901', 'outra@email.com');
linha 2: ERROR:  duplicate key value violates unique constraint "uk_cliente_cpf"
DETAIL:  Key (cpf)=(12345678901) already exists.
```

Por que foi recusado: O CPF 12345678901 já é da cliente Maria Souza. **O banco recusou corretamente.**

[T04] Código de barras repetido

Regra de negócio: RNF05: o código de barras localiza um único produto no caixa.
Restrição: `uk_produto_barras (UNIQUE)`
Comando e resultado:

psql — petly_club (PostgreSQL 16) — executado em 27/10/2026

```
petly_club=# INSERT INTO produto (nome, marca, codigo_barras, preco, controla_validade, id_categoria)
petly_club-# VALUES ('Racao Copia', 'Marca X', '7890000000011', 99.90, TRUE, 1);
linha 2: ERROR:  duplicate key value violates unique constraint "uk_produto_barras"
DETAIL:  Key (codigo_barras)=(7890000000011) already exists.
```

Por que foi recusado: O código 7890000000011 já é da Ração Premium 15kg. **O banco recusou corretamente.**

2. Chave estrangeira (integridade referencial)

[T05] Pedido de cliente inexistente

Regra de negócio: RN01, RP02: todo pedido pertence a um cliente cadastrado.
Restrição: `fk_pedido_cliente (FOREIGN KEY)`
Comando e resultado:

psql — petly_club (PostgreSQL 16) — executado em 27/10/2026

```
petly_club=# INSERT INTO pedido (canal_venda, tipo_recebimento, id_cliente)
petly_club-# VALUES ('loja_virtual', 'entrega', 999);
linha 2: ERROR:  insert or update on table "pedido" violates foreign key constraint "fk_pedido_cliente"
DETAIL:  Key (id_cliente)=(999) is not present in table "cliente".
```

Por que foi recusado: Não existe cliente 999. **O banco recusou corretamente.**

[T06] Venda saindo do lote de outro produto

Regra de negócio: RN20 e ajuste [A1]: a unidade vendida sai de um lote do mesmo produto do item.
Restrição: `fk_mov_lote: FK composta (id_lote, id_produto) → lote`
Comando e resultado:

psql — petly_club (PostgreSQL 16) — executado em 27/10/2026

```
petly_club=# INSERT INTO movimentacao_estoque (tipo, quantidade, id_lote, id_produto, id_funcionario, id_pedido)
petly_club-# VALUES ('venda', 1, 3, 1, 2, 1);
linha 2: ERROR:  insert or update on table "movimentacao_estoque" violates foreign key constraint "fk_mov_lote"
DETAIL:  Key (id_lote, id_produto)=(3, 1) is not present in table "lote".
```

Por que foi recusado: O lote 3 é de Shampoo (produto 2), mas a venda é da Ração (produto 1). **O banco recusou corretamente.**

[T07] Excluir cliente que tem pedidos

Regra de negócio: RNF03: o histórico de vendas é mantido (5 anos); nada é apagado em cascata.
Restrição: `fk_pedido_cliente ON DELETE RESTRICT`
Comando e resultado:

psql — petly_club (PostgreSQL 16) — executado em 27/10/2026

```
petly_club=# DELETE FROM cliente WHERE id_cliente = 1;
linha 1: ERROR:  update or delete on table "cliente" violates foreign key constraint "fk_pedido_cliente" on table "pedido"
DETAIL:  Key (id_cliente)=(1) is still referenced from table "pedido".
```

Por que foi recusado: A cliente 1 tem o pedido 1; apagar o cadastro deixaria o pedido sem cliente. **O banco recusou corretamente.**

3. Domínio: NOT NULL e CHECK

[T08] Produto sem categoria

Regra de negócio: RN04: todo produto pertence a uma categoria.

Restrição: `produto.id_categoria NOT NULL`

Comando e resultado:

```
psql — petly_club (PostgreSQL 16) — executado em 27/10/2026

petly_club=# INSERT INTO produto (nome, marca, codigo_barras, preco, controla_validade)
petly_club-# VALUES ('Coleira P', 'Marca D', '7890000000999', 39.90, FALSE);
linha 2: ERROR:  null value in column "id_categoria" of relation "produto" violates not-null constraint
DETAIL:  Failing row contains (10, Coleira P, null, Marca D, 7890000000999, 39.90, 0, f, null).
```

Por que foi recusado: O produto foi enviado sem categoria. **O banco recusou corretamente.**

[T09] Preço zero

Regra de negócio: Dicionário: o preço de venda é maior que zero.

Restrição: `ck_produto_preco CHECK (preco > 0)`

Comando e resultado:

```
psql — petly_club (PostgreSQL 16) — executado em 27/10/2026

petly_club=# INSERT INTO produto (nome, marca, codigo_barras, preco, controla_validade, id_categoria)
petly_club-# VALUES ('Petisco Gratis', 'Marca C', '7890000000998', 0, TRUE, 1);
linha 2: ERROR:  new row for relation "produto" violates check constraint "ck_produto_preco"
DETAIL:  Failing row contains (12, Petisco Gratis, null, Marca C, 7890000000998, 0.00, 0, t, 1).
```

Por que foi recusado: Preço 0 não é aceito. **O banco recusou corretamente.**

[T10] Loja física com entrega

Regra de negócio: RN17: o caixa só vende no balcão; retirada e entrega são da loja virtual.

Restrição: `ck_pedido_canal_recebimento (CHECK)`

Comando e resultado:

```
psql — petly_club (PostgreSQL 16) — executado em 27/10/2026

petly_club=# INSERT INTO pedido (canal_venda, tipo_recebimento, id_cliente, id_funcionario)
petly_club-# VALUES ('loja_fisica', 'entrega', 1, 2);
linha 2: ERROR:  new row for relation "pedido" violates check constraint "ck_pedido_canal_recebimento"
DETAIL:  Failing row contains (10, 1, 2, 2026-10-27 22:45:59.441162, loja_fisica, entrega, pendente, 0.00, null, null).
```

Por que foi recusado: A combinação loja_fisica + entrega não existe no negócio. **O banco recusou corretamente.**

[T11] Venda no caixa sem funcionário

Regra de negócio: RN14: toda venda da loja física é registrada por um funcionário.

Restrição: `ck_pedido_func_fisica (CHECK)`

Comando e resultado:

```
psql — petly_club (PostgreSQL 16) — executado em 27/10/2026

petly_club=# INSERT INTO pedido (canal_venda, tipo_recebimento, id_cliente)
petly_club-# VALUES ('loja_fisica', 'balcao', 1);
linha 2: ERROR:  new row for relation "pedido" violates check constraint "ck_pedido_func_fisica"
DETAIL:  Failing row contains (12, 1, null, 2026-10-27 22:45:59.537361, loja_fisica, balcao, pendente, 0.00, null, null).
```

Por que foi recusado: Falta o funcionário que registrou a venda. **O banco recusou corretamente.**

[T12] Cancelamento no caixa sem autorização

Regra de negócio: RP07 e ajuste [A3]: cancelar na loja física exige autorização da gerência.

Restrição: `ck_pedido_cancel_fisica (CHECK)`

Comando e resultado:

psql — petly_club (PostgreSQL 16) — executado em 27/10/2026

```
petly_club=# UPDATE pedido SET status = 'cancelado'
petly_club-# WHERE id_pedido = 1;
linha 2: ERROR:  new row for relation "pedido" violates check constraint "ck_pedido_cancel_fisica"
DETAIL:  Failing row contains (1, 1, 2, 2026-10-20 10:30:00, loja_fisica, balcao, cancelado, 0.00, null, null).
```

Por que foi recusado: O pedido 1 é da loja física e não tem id_autorizador_cancelamento. **O banco recusou corretamente.**

[T13] Perda sem autorizador

Regra de negócio: RP11 e ajuste [A2]: perda e ajuste de estoque só com autorização da gerência.

Restrição: `ck_mov_autorizacao (CHECK)`

Comando e resultado:

psql — petly_club (PostgreSQL 16) — executado em 27/10/2026

```
petly_club=# INSERT INTO movimentacao_estoque (tipo, quantidade, motivo, id_lote, id_produto, id_funcionario)
petly_club-# VALUES ('perda', 1, 'Embalagem rasgada', 2, 1, 3);
linha 2: ERROR:  new row for relation "movimentacao_estoque" violates check constraint "ck_mov_autorizacao"
DETAIL:  Failing row contains (32, 2, 1, perda, 1, 2026-10-27 22:45:59.706531, Embalagem rasgada, 3, null, null, null).
```

Por que foi recusado: A perda foi registrada sem id_autorizador. **O banco recusou corretamente.**

[T14] Perda sem motivo

Regra de negócio: RP11, RF35: perda, ajuste e devolução precisam de motivo.

Restrição: `ck_mov_motivo (CHECK)`

Comando e resultado:

psql — petly_club (PostgreSQL 16) — executado em 27/10/2026

```
petly_club=# INSERT INTO movimentacao_estoque (tipo, quantidade, id_lote, id_produto, id_funcionario, id_autorizador)
petly_club-# VALUES ('perda', 1, 2, 1, 3, 1);
linha 2: ERROR:  new row for relation "movimentacao_estoque" violates check constraint "ck_mov_motivo"
DETAIL:  Failing row contains (34, 2, 1, perda, 1, 2026-10-27 22:45:59.804024, null, 3, 1, null, null).
```

Por que foi recusado: O motivo ficou vazio. **O banco recusou corretamente.**

[T15] Crédito em 4 vezes

Regra de negócio: RN24: crédito em até 3x sem juros.

Restrição: `ck_pag_parcelas CHECK (numero_parcelas BETWEEN 1 AND 3)`

Comando e resultado:

psql — petly_club (PostgreSQL 16) — executado em 27/10/2026

```
petly_club=# INSERT INTO pagamento (id_pedido, forma_pagamento, valor, numero_parcelas)
petly_club-# VALUES (2, 'cartao_credito', 134.70, 4);
linha 2: ERROR:  new row for relation "pagamento" violates check constraint "ck_pag_parcelas"
DETAIL:  Failing row contains (9, 2, cartao_credito, 134.70, 4, null, pendente).
```

Por que foi recusado: 4 parcelas passa do limite. **O banco recusou corretamente.**

[T16] Débito parcelado

Regra de negócio: RN24: só o cartão de crédito é parcelado; as demais formas são em 1 vez.
Restrição: `ck_pag_parcelas_credito (CHECK)`
Comando e resultado:

psql — petly_club (PostgreSQL 16) — executado em 27/10/2026

```
petly_club=# INSERT INTO pagamento (id_pedido, forma_pagamento, valor, numero_parcelas)
petly_club-# VALUES (2, 'cartao_debito', 134.70, 2);
linha 2: ERROR:  new row for relation "pagamento" violates check constraint "ck_pag_parcelas_credito"
DETAIL:  Failing row contains (11, 2, cartao_debito, 134.70, 2, null, pendente).
```

Por que foi recusado: Débito em 2 parcelas não existe. **O banco recusou corretamente.**

[T17] Conta recebida sem data

Regra de negócio: RN24: o valor só é recebido quando entra na conta, e essa data fica registrada.
Restrição: `ck_cr_recebimento (CHECK)`
Comando e resultado:

psql — petly_club (PostgreSQL 16) — executado em 27/10/2026

```
petly_club=# UPDATE conta_receber SET status = 'recebida'
petly_club-# WHERE id_conta_receber = 2;
linha 2: ERROR:  new row for relation "conta_receber" violates check constraint "ck_cr_recebimento"
DETAIL:  Failing row contains (2, 2, 1, 44.90, 42.65, 2026-11-24, null, recebida).
```

Por que foi recusado: A parcela foi marcada como recebida sem data_recebimento. **O banco recusou corretamente.**

4. Tabelas associativas (PK composta)

[T18] Mesmo produto duas vezes no pedido

Regra de negócio: RN03: o produto aparece uma única vez em cada pedido, com a quantidade total.
Restrição: `pk_item_pedido PRIMARY KEY (id_pedido, id_produto)`
Comando e resultado:

psql — petly_club (PostgreSQL 16) — executado em 27/10/2026

```
petly_club=# INSERT INTO item_pedido (id_pedido, id_produto, quantidade, preco_unitario)
petly_club-# VALUES (1, 1, 3, 189.90);
linha 2: ERROR:  duplicate key value violates unique constraint "pk_item_pedido"
DETAIL:  Key (id_pedido, id_produto)=(1, 1) already exists.
```

Por que foi recusado: A Ração (produto 1) já é item do pedido 1. **O banco recusou corretamente.**

[T19] Mesmo produto duas vezes na compra

Regra de negócio: RN10: o produto aparece uma vez por compra, com a quantidade total.
Restrição: `pk_item_compra PRIMARY KEY (id_compra, id_produto)`
Comando e resultado:

psql — petly_club (PostgreSQL 16) — executado em 27/10/2026

```
petly_club=# INSERT INTO item_compra (id_compra, id_produto, quantidade_comprada, custo_unitario)
petly_club-# VALUES (1, 1, 5, 100.00);
linha 2: ERROR:  duplicate key value violates unique constraint "pk_item_compra"
DETAIL:  Key (id_compra, id_produto)=(1, 1) already exists.
```

Por que foi recusado: A Ração (produto 1) já é item da compra 1. **O banco recusou corretamente.**

5. Gatilhos

[T20] Alterar uma movimentação de estoque

Regra de negócio: RN19, RNF03: o histórico do estoque não pode ser alterado nem excluído; erro se corrige com ajuste.

Restrição: `trg01_mov_imutavel (gatilho BEFORE UPDATE OR DELETE)`

Comando e resultado:

```
psql — petly_club (PostgreSQL 16) — executado em 27/10/2026
petly_club=# UPDATE movimentacao_estoque SET quantidade = 99
petly_club-# WHERE id_movimentacao = 1;
linha 2: ERROR:  Movimentacao de estoque nao pode ser alterada nem excluida (RN19). Registre um ajuste.
CONTEXT:  PL/pgSQL function fn_mov_imutavel() line 3 at RAISE
```

Por que foi recusado: O gatilho bloqueia qualquer UPDATE ou DELETE na tabela. **O banco recusou corretamente.**

[T21] Vender sem saldo

Regra de negócio: RN07: não se vende mais do que a quantidade disponível do lote.

Restrição: `trg02_mov_saldo (gatilho BEFORE INSERT)`

Comando e resultado:

```
psql — petly_club (PostgreSQL 16) — executado em 27/10/2026
petly_club=# INSERT INTO movimentacao_estoque (tipo, quantidade, id_lote, id_produto, id_funcionario, id_pedido)
petly_club-# VALUES ('venda', 1, 6, 4, 2, 1);
linha 2: ERROR:  Saldo insuficiente no lote 6 (RN07): o saldo fisico ficaria em -1.
CONTEXT:  PL/pgSQL function fn_mov_saldo() line 30 at RAISE
```

Por que foi recusado: O lote 6 (brinquedo) já tem saldo 0: a última unidade foi vendida no pedido 1. **O banco recusou corretamente.**

[T22] Entrada em lote vencido

Regra de negócio: RF12, RP09: lote vencido só admite perda, liberação de reserva e ajuste.

Restrição: `trg03_mov_validade (gatilho BEFORE INSERT)`

Comando e resultado:

```
psql — petly_club (PostgreSQL 16) — executado em 27/10/2026
petly_club=# INSERT INTO movimentacao_estoque (tipo, quantidade, data_hora, id_lote, id_produto, id_funcionario, id_compra)
petly_club-# VALUES ('entrada', 5, '2026-10-27 10:00', 4, 3, 3, 101);
linha 2: ERROR:  Lote 4 vencido em 2026-09-30 (RF12): entrada nao permitida.
CONTEXT:  PL/pgSQL function fn_mov_validade() line 8 at RAISE
```

Por que foi recusado: O lote 4 venceu em 30/09/2026. **O banco recusou corretamente.**

[T23] Pagar mais que o valor da parcela

Regra de negócio: RN23: a soma dos pagamentos não passa do valor da parcela.

Restrição: `trg08_pagconta_valida (gatilho BEFORE INSERT)`

Comando e resultado:

```
psql — petly_club (PostgreSQL 16) — executado em 27/10/2026
petly_club=# INSERT INTO pagamento_conta (id_conta_pagar, valor_pago, forma_pagamento)
petly_club-# VALUES (1, 600.00, 'pix');
linha 2: ERROR:  Pagamento acima do valor da parcela 1 (RN23): ja pago 500.00, novo 600.00, valor 1000.00.
CONTEXT:  PL/pgSQL function fn_pagconta_valida() line 18 at RAISE
```

Por que foi recusado: A parcela 1 vale R$ 1.000,00 e já tem R$ 500,00 pagos. **O banco recusou corretamente.**

6. Concorrência

[T24] Duas vendas simultâneas da última unidade (cenário 1 do README)

Regra de negócio: RN07 e RNF10: duas vendas não podem usar a mesma unidade; o saldo nunca fica negativo.

Restrição: `trg02_mov_saldo`, que trava a linha do lote (SELECT ... FOR UPDATE) antes de conferir o saldo.

Preparação: numa cópia temporária da base, o lote 6 (brinquedo) recebe 1 unidade por ajuste autorizado, e um cliente abre o pedido 7 na loja virtual com 1 brinquedo.

psql — petly_club (PostgreSQL 16) — executado em 27/10/2026

```
petly_club=# -- Preparação (base temporária): o lote 6 (brinquedo) recebe 1 unidade por ajuste autorizado,
petly_club=# -- e um cliente abre o pedido 7 na loja virtual com 1 brinquedo
petly_club=# INSERT INTO movimentacao_estoque (tipo, quantidade, motivo, id_lote, id_produto, id_funcionario,
id_autorizador)
petly_club-# VALUES ('ajuste_positivo', 1, 'Contagem encontrou 1 unidade', 6, 4, 3, 1);
INSERT 0 1
petly_club=# INSERT INTO pedido (id_pedido, canal_venda, tipo_recebimento, id_cliente) OVERRIDING SYSTEM VALUE
petly_club-# VALUES (7, 'loja_virtual', 'entrega', 3);
INSERT 0 1
petly_club=# INSERT INTO item_pedido (id_pedido, id_produto, quantidade, preco_unitario) VALUES (7, 4, 1, 19.90);
INSERT 0 1
petly_club=# SELECT * FROM v_lote_saldo WHERE id_lote = 6;
 id_lote | id_produto | saldo_fisico | quantidade_reservada | quantidade_disponivel
petly_club=# ---------+------------+--------------+----------------------+----------------------
       6 |          4 |            1 |                    0 |                    1
(1 row)
```

Controle: sozinha, a reserva da loja virtual seria aceita (desfeita com ROLLBACK).

psql — petly_club (PostgreSQL 16) — executado em 27/10/2026

```
petly_club=# -- Controle: sem a venda do caixa, a mesma reserva seria aceita
petly_club=# BEGIN;
BEGIN
petly_club=# INSERT INTO movimentacao_estoque (tipo, quantidade, id_lote, id_produto, id_pedido)
petly_club-# VALUES ('reserva', 1, 6, 4, 7);
INSERT 0 1
petly_club=# ROLLBACK;
ROLLBACK
```

Sessão 1 — caixa: vende a unidade e mantém a transação aberta por 3 segundos.

psql — petly_club (PostgreSQL 16) — executado em 27/10/2026

```
petly_club=# BEGIN;
BEGIN
petly_club=# INSERT INTO movimentacao_estoque (tipo, quantidade, id_lote, id_produto, id_funcionario, id_pedido)
petly_club-# VALUES ('venda', 1, 6, 4, 2, 1);
INSERT 0 1
petly_club=# SELECT pg_sleep(3);  -- o caixa ainda não terminou a venda
 pg_sleep
petly_club=# ----------

(1 row)

petly_club=# COMMIT;
COMMIT
```

Sessão 2 — loja virtual, iniciada 0,5 s depois, enquanto o caixa ainda não tinha terminado. Ela ficou esperando a trava e só respondeu quando o caixa fez COMMIT (cerca de 2,5 s depois), já com o saldo atualizado:

psql — petly_club (PostgreSQL 16) — executado em 27/10/2026

```
petly_club=# INSERT INTO movimentacao_estoque (tipo, quantidade, id_lote, id_produto, id_pedido)
petly_club-# VALUES ('reserva', 1, 6, 4, 7);
linha 2: ERROR:  Quantidade indisponivel no lote 6 (RN07): reservado 1 maior que o saldo 0.
CONTEXT:  PL/pgSQL function fn_mov_saldo() line 36 at RAISE
```

Situação final do lote:

psql — petly_club (PostgreSQL 16) — executado em 27/10/2026

```
petly_club=# SELECT * FROM v_lote_saldo WHERE id_lote = 6;
 id_lote | id_produto | saldo_fisico | quantidade_reservada | quantidade_disponivel
petly_club=# ---------+------------+--------------+----------------------+-----------------------
       6 |          4 |            0 |                    0 |                     0
(1 row)
```

Resultado: a primeira venda foi aceita e a segunda, recusada. O saldo terminou em 0. **O banco tratou a concorrência corretamente.**

Parte 2 — UPDATE e DELETE que o banco deve aceitar

Estes comandos mostram operações normais do dia a dia. Eles são aceitos porque não violam nenhuma regra, e o resultado mostra que as restrições continuam valendo depois da alteração.

[U01] UPDATE: reajuste de preço não altera vendas antigas

Regra de negócio: RN13: o preço vendido fica registrado no item do pedido.

Restrição envolvida: `item_pedido.preco_unitario guarda o preço do momento da venda`

Comando e resultado:

psql — petly_club (PostgreSQL 16) — executado em 27/10/2026

```
petly_club=# BEGIN;
BEGIN
petly_club=# UPDATE produto SET preco = 199.90 WHERE id_produto = 1;
UPDATE 1
petly_club=# SELECT p.preco AS preco_atual, i.preco_unitario AS preco_no_pedido_1
petly_club-# FROM produto p JOIN item_pedido i ON i.id_produto = p.id_produto
petly_club-# WHERE p.id_produto = 1 AND i.id_pedido = 1;
 preco_atual | preco_no_pedido_1
petly_club=# -------------+-------------------
       199.90 |            189.90
(1 row)

petly_club=# ROLLBACK;
ROLLBACK
```

O que o comando faz: O preço de catálogo passou para R$ 199,90, mas o pedido 1 continua com R$ 189,90. **Aceito, como esperado.**

[U02] UPDATE: pedido entregue

Regra de negócio: RN16, RN17: o status do pedido e o da entrega andam juntos; entrega concluída tem data.

Restrição envolvida: `ck_pedido_status_fluxo e ck_entrega_data`

Comando e resultado:

psql — petly_club (PostgreSQL 16) — executado em 27/10/2026

```
petly_club=# BEGIN;
BEGIN
petly_club=# UPDATE pedido SET status = 'entregue' WHERE id_pedido = 2;
UPDATE 1
petly_club=# UPDATE entrega SET status = 'entregue', data_entrega = '2026-10-28'
petly_club-# WHERE id_pedido = 2;
UPDATE 1
petly_club=# SELECT p.status AS pedido, e.status AS entrega, e.data_entrega
petly_club-# FROM pedido p JOIN entrega e USING (id_pedido) WHERE id_pedido = 2;
  pedido  | entrega  | data_entrega
petly_club=# ----------+----------+--------------
 entregue | entregue | 2026-10-28
(1 row)

petly_club=# ROLLBACK;
ROLLBACK
```

O que o comando faz: Os dois status passam para 'entregue', e a data fica registrada. **Aceito, como esperado.**

[U03] INSERT + gatilho: pagamento que quita a parcela

Regra de negócio: RN23: a parcela só fica paga quando a soma dos pagamentos chega ao valor.
Restrição envolvida: `trg08_pagconta_status (gatilho AFTER INSERT)`
Comando e resultado:

psql — petly_club (PostgreSQL 16) — executado em 27/10/2026

```
petly_club=# BEGIN;
BEGIN
petly_club=# INSERT INTO pagamento_conta (id_conta_pagar, valor_pago, forma_pagamento)
petly_club-# VALUES (1, 500.00, 'pix');
INSERT 0 1
petly_club=# SELECT * FROM v_conta_pagar_saldo WHERE id_conta_pagar = 1;
 id_conta_pagar | id_compra | numero_parcela |  valor  | data_vencimento | status | valor_pago_total | saldo_devedor
petly_club=# ----------------+-----------+----------------+---------+-----------------+--------+------------------+---------
------
              1 |         1 |              1 | 1000.00 | 2026-11-22      | paga   |          1000.00 |          0.00
(1 row)

petly_club=# ROLLBACK;
ROLLBACK
```

O que o comando faz: Com os R$ 500,00 restantes, o gatilho muda a parcela de 'paga_parcialmente' para 'paga' e o saldo devedor vai a zero. **Aceito, como esperado.**

[D01] DELETE: categoria sem produtos

Regra de negócio: Uma categoria criada por engano pode ser excluída enquanto não tiver produtos.
Restrição envolvida: `fk_produto_categoria ON DELETE RESTRICT (só bloqueia se houver produto)`
Comando e resultado:

psql — petly_club (PostgreSQL 16) — executado em 27/10/2026

```
petly_club=# BEGIN;
BEGIN
petly_club=# INSERT INTO categoria (nome) VALUES ('Categoria Teste');
INSERT 0 1
petly_club=# DELETE FROM categoria WHERE nome = 'Categoria Teste';
DELETE 1
petly_club=# ROLLBACK;
ROLLBACK
```

O que o comando faz: A exclusão é aceita porque nenhum produto aponta para a categoria. Já a categoria 1, que tem produtos, seria recusada como no T07. **Aceito, como esperado.**

Limites encontrados

As restrições do banco cobrem as regras que dependem de uma linha (CHECK), da existência de outro registro (FK) ou de unicidade (PK e UNIQUE). Os gatilhos TRG-01, TRG-02, TRG-03 e TRG-08 cobrem as regras que dependem do histórico ou de somas. As demais regras da seção 6 do Modelo Lógico, por exemplo o desconto acima do limite (RP05), a exigência de e-mail para comprar na loja virtual e a cópia automática do preço para o item do pedido (RN13), ficam a cargo da aplicação, dentro da mesma transação da operação.